

CONSTRUCTION INTELLIGENT HUB

An AI-Assisted Construction Site Management Platform

Project Documentation Report

Prepared from source-code analysis of the "Hemanthinfo" project repository

Engineering Project Report

1. Project Title

Construction Intelligent Hub

Internally, the FastAPI backend describes itself as "Backend API services for AI Powered Construction Management & Intelligent Hiring Platform" (see backend/main.py). The project folder and packaged application are named "Hemanthinfo". Only the construction-management functionality is implemented in the current codebase; hiring/recruitment features referenced in the description are not present in the code and are noted as a gap in Section 19 (Disadvantages) and Section 22 (Future Enhancements).

2. Introduction

2.1 Background

Construction sites generate large volumes of operational data every day — equipment usage, environmental readings, safety incidents, material stock levels, and project schedules. In most small and mid-sized construction firms this data is still tracked manually across spreadsheets, paper checklists, and radio/phone communication. Construction Intelligent Hub is a full-stack web application that centralises this information into a single dashboard and augments it with AI-assisted document analysis, an AI construction assistant chatbot, and simulated IoT-style sensor monitoring.

2.2 Purpose

The purpose of the project is to demonstrate how a modern Python-based stack (FastAPI + Streamlit) can be combined with a generative-AI service (Google Gemini) to give construction personnel a single place to: authenticate, view a role-aware dashboard, log and simulate site sensor readings (air quality, noise, water, structural health, weather, energy), track inventory and equipment telematics, forecast schedule risk, analyse uploaded project documents with a Retrieval-Augmented Generation (RAG) pipeline, and converse with an AI assistant capable of text, voice and image input.

2.3 Current Industry Need

The construction industry has historically lagged behind other sectors in digital adoption. Safety incidents, environmental non-compliance, material wastage, and schedule overruns are frequently discovered late because monitoring is manual and reactive rather than continuous and proactive. There is a recognised need for lightweight, role-based digital dashboards that give site staff and management real-time visibility without requiring expensive, hardware-heavy IoT deployments.

2.4 Importance

This project is important as an academic and practical demonstration of how AI services (LLMs, embeddings, vector search) can be integrated into a traditional CRUD web application without requiring a dedicated data-science team, using widely available managed APIs (Google Gemini) and open-source Python libraries.

2.5 Scope

The implemented scope covers: user authentication with email OTP verification, a role-aware dashboard with tasks/activity feed/notifications, twelve site-monitoring modules (air quality, noise, water, structural health, weather, energy — each with logging, simulation and configuration endpoints), material inventory and reordering, construction equipment telematics, a CPM-style schedule risk predictor, a safety incident/checklist module, a simulated PPE-compliance image-analysis endpoint, a document analyzer with Gemini-based summarisation and FAISS-based RAG question answering, and a general-purpose AI construction assistant supporting text, image and voice (speech-to-text/text-to-speech) interaction.

Out of scope / not implemented: real hardware IoT sensor integration (all sensor data is simulated with randomised values), the hiring/recruitment module implied by the backend description, a production MongoDB/Redis deployment (both are supported in code but the project runs on local SQLite by default), and enforced role-based access control (the user's role personalises content but does not restrict endpoint access in the current code).

2.6 Target Users

The application defines eleven role types in its data model — Admin, Engineer, Contractor, Worker, HR, Client, Supplier, Project Manager, Safety Officer, Site Supervisor and Volunteer — and every authenticated user can select one of these roles at registration. The dashboard and chatbot responses are personalised per role, making the primary target users construction site engineers, project managers, safety officers and site supervisors who need a consolidated operational view.

3. Problem Statement

3.1 Existing Problems

- Site conditions (air quality, noise, water leakage, structural stress, weather hazards, energy usage) are tracked manually or not tracked at all in small construction firms.
- Material inventory is often maintained on spreadsheets, leading to stock-outs or over-ordering.
- Project documents (contracts, reports, compliance certificates) are difficult to search; finding a specific clause means manually re-reading PDFs.
- Safety checklists and incident logs are paper-based, making trend analysis and audit difficult.
- Schedule delays are usually identified after they occur rather than being forecast in advance.

3.2 Current Limitations

Existing spreadsheet- and paper-based workflows do not provide a single, real-time, role-aware view of a site's status. They also cannot answer natural-language questions about stored documents or offer AI-assisted first-pass analysis of uploaded reports.

3.3 Why This Project Is Required

A unified web platform reduces the friction of switching between multiple disconnected tools (email, spreadsheets, paper forms, messaging apps) and introduces AI assistance for tasks — such as document summarisation and question answering — that would otherwise require manual reading.

3.4 Real-World Problems Solved

- Centralised, authenticated dashboard for tasks, notifications and activity history.
- Structured logging and configurable safety thresholds for six environmental/structural monitoring domains.
- Automatic reorder suggestions when material stock falls below a threshold.
- AI-generated document summaries and a RAG-based question-answering chat over uploaded project files.
- A conversational AI assistant that accepts text, uploaded images/documents, and voice input, and can respond with synthesised speech.

4. Objectives

4.1 Primary Objectives

1. Build a secure authentication system with JWT-based sessions and email OTP verification for account activation and password reset.
2. Provide a role-aware operational dashboard summarising tasks, activity logs and notifications for the logged-in user.
3. Implement site-monitoring modules — air quality, noise, water, structural health, weather hazards and energy consumption — each supporting reading logs, configurable safety thresholds, and data simulation for demonstration purposes.
4. Implement inventory management with stock tracking, consumption logging and automatic reorder workflows.
5. Implement equipment telematics tracking with maintenance scheduling.
6. Implement a CPM-style schedule task tracker with basic delay-risk prediction.
7. Integrate a generative-AI (Gemini) service for a general construction assistant chatbot, a safety chatbot, and document analysis/summarisation.
8. Implement a Retrieval-Augmented Generation pipeline (FAISS + Sentence-Transformers) so uploaded documents can be queried in natural language.

4.2 Secondary Objectives

9. Support voice interaction (speech-to-text input and text-to-speech output) in the AI assistant.
10. Provide a simulated computer-vision endpoint for PPE (personal protective equipment) compliance checking from an uploaded image.
11. Provide automated pytest coverage across authentication, dashboard, and all monitoring modules.
12. Support both SQLite (development) and MongoDB (production-oriented) data backends, with an optional Redis cache layer.

5. Proposed System

5.1 Architecture

The system follows a two-tier client/service architecture. The Streamlit frontend (frontend/app.py and frontend/pages/*.py) is a server-rendered Python UI that never talks to the database directly — every action is routed through frontend/utils/api_client.py, which issues authenticated HTTP requests (using the Python requests library) to the FastAPI backend (backend/main.py) running on a separate port. The backend exposes REST endpoints grouped into routers (one per feature area), each backed by a service module containing the business logic, and a SQLAlchemy model persisted to SQLite by default.

5.2 Working Methodology

- The Streamlit app is launched on port 8501 and the FastAPI app on port 8000 (run.bat starts it on 8005 in the provided startup script); both must run concurrently.
- On startup, the FastAPI lifespan handler calls init_db(), which creates all SQLAlchemy tables from the imported models and ensures the ./data/uploads and ./data/faiss_indexes directories exist.
- CORS middleware is configured to allow the Streamlit origins (localhost:8501/8000 and their 127.0.0.1 equivalents) to call the API.

5.3 User Flow

13. A new user registers via the /auth/register endpoint, providing email, name, password and a role selected from the eleven predefined roles.
14. An OTP is generated and (in a configured environment) emailed via SMTP; the user verifies the OTP through /auth/verify-otp before the account is treated as active.
15. The user logs in via /auth/login, receiving a JWT bearer token that the Streamlit session stores and attaches to every subsequent API call.
16. The user lands on the role-aware dashboard, then navigates via the sidebar to individual monitoring, inventory, schedule, safety, telematics, or AI assistant pages.

5.4 AI Workflow

Two distinct AI workflows are implemented. First, a direct-prompt workflow (used by the general assistant and safety chatbot) sends the user's message, optional uploaded image, and role context directly to the Google Gemini API via the google-generativeai SDK and returns the model's text response, optionally converted to speech with gTTS. Second, a Retrieval-Augmented Generation workflow (used by the document analyzer) extracts text from an uploaded PDF, DOCX or XLSX file, splits and embeds it with a Sentence-Transformers model, indexes the vectors in a FAISS index on disk, and — on a user query — retrieves the most relevant chunks before asking Gemini to answer using only that retrieved context.

5.5 Data Flow

Frontend page → api_client.py (adds JWT header) → FastAPI router → Pydantic schema validation → service-layer function → SQLAlchemy model → SQLite database → JSON response → rendered back into the Streamlit page (typically as a Plotly chart, table, or metric card).

5.6 Backend Flow

Each monitoring router (e.g. `backend/routers/air_quality.py`) follows a consistent pattern: GET `/logs` and `/configs` read historical readings and threshold configuration for the authenticated user's context; POST `/log` records a manually entered reading; POST `/simulate` generates a batch of randomised but plausible sensor readings (using Python's `random` module) to populate the dashboard for demonstration; POST `/config` updates the safety threshold; and POST `/reset` restores default configuration values.

5.7 Frontend Flow

`frontend/app.py` owns the Streamlit session state (authentication flag, JWT token, active role, user profile) and renders the login/registration/OTP screens before handing off to the page router. Once authenticated, each page module under `frontend/pages/` independently calls the API client to fetch and post data, and renders it using Streamlit widgets and Plotly charts, with shared styling loaded from `frontend/assets/style.css`.

6. Existing System

6.1 Current Methods

In the absence of a platform like this, construction teams typically rely on a combination of paper safety checklists, WhatsApp/phone communication for incident reporting, Excel spreadsheets for inventory and scheduling, and email for document circulation.

6.2 Drawbacks

- No single source of truth — data is scattered across tools and people.
- No automated alerting when a safety threshold (noise, air quality, structural stress) is exceeded.
- Manual document search is slow and error-prone.
- Reordering materials depends on someone noticing low stock rather than a systematic threshold check.

6.3 Limitations

Spreadsheet-based tracking does not scale well with multiple simultaneous sites or shifts, and provides no role-based personalisation or historical trend visualisation.

6.4 Manual Process

A typical manual process requires a site supervisor to physically inspect conditions, record them on paper, and later transcribe them into a spreadsheet or report — introducing delay and transcription error.

6.5 Comparison with Proposed System

Aspect	Existing (Manual) System	Proposed System (Construction Intelligent Hub)
Data storage	Spreadsheets / paper	Centralised SQLite/MongoDB database
Monitoring	Manual inspection	Loggable + simulated readings with configurable thresholds
Document search	Manual re-reading	AI-powered RAG search and Gemini summarisation
Access	Local files, single device	Web-based, authenticated, multi-page dashboard
Alerts	None / verbal	Threshold configuration per monitoring module
Assistant	None	Gemini-based text/image/voice AI assistant

7. Technology Stack

The following table lists technologies detected in requirements.txt and confirmed to be imported and used in the source code (see Section 19 for dependencies that are listed but not actually wired into the code).

Technology	Version	Purpose
Python	3.11 (per compiled .pyc tags)	Primary programming language for backend and frontend
FastAPI	0.111.0	REST API backend framework
Uvicorn	0.29.0	ASGI server running the FastAPI app
Streamlit	1.35.0	Server-rendered Python frontend framework
SQLAlchemy	2.0.30	ORM for the SQLite relational database
Motor	3.4.0	Async MongoDB driver (production-oriented data layer)
Redis (redis-py)	5.0.4	Optional cache layer, degrades gracefully if unavailable
python-jose	3.3.0	JWT creation/verification for authentication
passlib + bcrypt	1.7.4 / 4.1.3	Password hashing
pydantic / pydantic-settings	2.7.1 / 2.3.4	Request/response validation and settings management
google-generativeai (Gemini)	0.5.4	LLM used for chat, safety chatbot and document summarisation
faiss-cpu	1.8.0	Vector similarity search for the RAG pipeline
sentence-transformers	3.0.1	Text embedding model (all-MiniLM-L6-v2) for RAG
PyPDF2 / python-docx / openpyxl	3.0.1 / 1.1.2 / 3.1.2	Text extraction from uploaded PDF, Word and Excel files
Pillow (PIL)	10.3.0	Image handling and simulated PPE-detection bounding-box drawing
SpeechRecognition	3.10.4	Speech-to-text transcription of uploaded audio
gTTS	2.5.1	Text-to-speech synthesis for AI assistant replies
Plotly	5.22.0	Interactive charts in the Streamlit dashboard
Pandas / NumPy	2.2.2 / 1.26.4	Data manipulation for monitoring and analytics
python-dotenv	1.0.1	Loading environment variables from .env
pytest / pytest-asyncio / httpx	8.2.1 / 0.23.7 / 0.27.0	Automated backend testing

8. Software Requirements

8.1 Software

Category	Requirement
Operating System	Windows 10/11 (a run.bat startup script is provided), also compatible with Linux/macOS
IDE	Any Python-capable IDE (e.g. VS Code, PyCharm)
Programming Language	Python 3.11
Backend Framework	FastAPI with Uvicorn ASGI server
Frontend Framework	Streamlit
Database	SQLite (development, default) or MongoDB (production-oriented)
Cache (optional)	Redis
Browser	Any modern browser (Chrome, Edge, Firefox) to access the Streamlit UI
Package Manager	pip, with dependencies listed in requirements.txt
Version Control	Git / GitHub (project shows a .github/workflows directory for CI)

8.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz or higher
RAM	4 GB	8 GB or higher (embedding model and FAISS index loaded in memory)
Storage	2 GB free space	5 GB+ free space (for FAISS indexes, audio cache, and logs)
Network	Internet connection required for Gemini API calls	Stable broadband connection

9. Why Streamlit?

Streamlit was selected as the frontend framework for this project for several practical reasons evident in the codebase's structure:

- **Rapid development:** each feature page (e.g. `frontend/pages/air_quality.py`) is a self-contained Python script with no separate HTML/CSS/JS build step, which speeds up iteration for a data- and AI-heavy dashboard.
- **Native Python integration:** Streamlit pages call the same Python API client, Pandas and Plotly libraries used elsewhere in the stack without a context switch to JavaScript.
- **Built-in interactive widgets:** forms, file uploaders, and session state are used extensively for login, document upload, and simulation triggers without hand-written JavaScript event handlers.
- **Data visualisation:** Plotly integrates directly into Streamlit for the charts used across every monitoring page.
- **AI application support:** Streamlit's simplicity makes it well suited to prototyping AI-assisted tools (chat interfaces, file analysis) quickly, as seen in the `construction_assistant.py` and `document_analyzer.py` pages.
- **Deployment simplicity:** a Streamlit app is a single Python process (`python -m streamlit run frontend/app.py`), simplifying local demonstration, as reflected in `run.bat`.
- **User-friendliness:** Streamlit's default component styling, extended here with custom CSS (`frontend/assets/style.css`), provides a clean UI without requiring dedicated frontend engineering.

9.1 Comparison with Flask, Django and React

Framework	Typical Use Case	Trade-off vs. Streamlit (in this project's context)
Flask	Lightweight custom web apps with full HTML control	Would require hand-written templates/JS for the same dashboards; more flexible but slower to build
Django	Large, batteries-included web applications with admin panels	Heavier framework with more boilerplate than needed for a data-dashboard-style app
React	Rich, highly interactive single-page applications	Requires a separate JS/TS build pipeline and a REST/GraphQL contract; more suited to production-grade UI than a rapid internal tool
Streamlit (chosen)	Data apps, dashboards, AI prototypes	Fast to build in pure Python; less flexible for pixel-perfect custom UI than React

10. System Modules

The modules below were identified from the actual backend routers, services, models and frontend pages present in the repository.

10.1 Authentication Module

Purpose: Handles user registration, OTP-based email verification, login, logout, and password reset.

Inputs: Email, name, password, role at registration; email + password at login; OTP code at verification.

Outputs: JWT access token, user profile record, OTP delivery (email) and a developer-only endpoint to retrieve the OTP for testing.

Workflow: backend/routers/auth.py exposes /register, /verify-otp, /resend-otp, /login, /logout, /forgot-password, /reset-password and a /developer/get-otp helper endpoint; backend/services/auth_service.py generates and validates OTPs and hashes passwords via backend/utils/security.py.

Technologies: FastAPI, SQLAlchemy, python-jose (JWT), passlib/bcrypt, smtplib for OTP email delivery.

10.2 Dashboard Module

Purpose: Provides the authenticated user's home screen: summary statistics, tasks, activity log and a role-aware chat.

Inputs: Authenticated user context (role, id); new task/activity payloads.

Outputs: Dashboard statistics, task list, activity feed, and a short role-personalised chat reply.

Workflow: backend/routers/dashboard.py exposes /stats, /tasks (POST/PUT), /activities (POST) and /chat; backend/services/dashboard_service.py computes role-specific statistics and notification visibility.

Technologies: FastAPI, SQLAlchemy (TaskItem, ActivityLog, SystemNotification models).

10.3 Environmental & Structural Monitoring Modules (Air Quality, Noise, Water, Structural Health, Weather, Energy)

Purpose: Six parallel modules that record and visualise site sensor readings and let the user configure safety thresholds.

Inputs: Sensor name and reading values (manual log) or a simulate request; safety threshold configuration values.

Outputs: Historical logs, current configuration, and a batch of newly generated simulated readings.

Workflow: Each module (e.g. backend/routers/air_quality.py + backend/models/air_quality.py) follows an identical pattern: GET /logs, GET /configs, POST /log, POST /simulate, POST /config, POST /reset. Simulated readings are generated with Python's random module rather than real sensor hardware.

Technologies: FastAPI, SQLAlchemy, Python random module, Plotly (frontend charts).

10.4 Inventory Management Module

Purpose: Tracks material stock levels and purchase/reorder workflow.

Inputs: Material consumption quantities, reorder requests, delivery status updates.

Outputs: Updated stock levels, list of pending/fulfilled orders.

Workflow: backend/routers/inventory.py exposes /stocks, /orders, /consume, /reorder, /update-delivery and /reset, backed by MaterialStock and MaterialOrder SQLAlchemy models.

Technologies: FastAPI, SQLAlchemy.

10.5 Equipment Telematics Module

Purpose: Tracks construction equipment health telemetry and maintenance scheduling.

Inputs: Equipment simulation trigger, maintenance scheduling requests.

Outputs: Equipment telemetry list (health indicators), updated maintenance schedule.

Workflow: backend/routers/telematics.py exposes /equipment, /simulate, /schedule-maintenance and /reset, backed by the EquipmentTelemetry model.

Technologies: FastAPI, SQLAlchemy.

10.6 Schedule Predictor Module

Purpose: Represents project tasks in a CPM-style structure and forecasts delay risk.

Inputs: Task list, a /predict trigger.

Outputs: Task list with computed risk/delay indicators.

Workflow: backend/routers/schedule.py exposes /tasks, /predict and /reset, backed by the ScheduleTask model; prediction logic lives in backend/services/schedule_service.py.

Technologies: FastAPI, SQLAlchemy.

10.7 Safety & Site-Safety Modules

Purpose: Covers incident logging, an AI safety chatbot, safety checklists and emergency SOPs, plus a separate simulated PPE-compliance image check.

Inputs: Incident reports, chat questions, uploaded site images.

Outputs: Logged incidents, chatbot answers, checklist/SOP text, and a compliance score with an annotated image for the PPE-detection endpoint.

Workflow: backend/routers/safety.py exposes /chat, /incidents, /checklist and /emergency-sop (Gemini-backed where configured); backend/routers/safety_monitoring.py exposes /detect, which calls backend/services/detection_service.py — this service simulates a computer-vision PPE detector using PIL to draw bounding boxes rather than a real trained model.

Technologies: FastAPI, SQLAlchemy, google-generativeai (Gemini), Pillow.

10.8 Document Analyzer / RAG Module

Purpose: Allows a user to upload a document, receive an AI-generated audit/summary, and ask natural-language questions answered from the document's content.

Inputs: Uploaded PDF/DOCX/XLSX file; natural-language questions about the document.

Outputs: Extracted text, Gemini-generated summary/risk analysis, and RAG-based question answers with retrieved source context.

Workflow: backend/routers/document_analyzer.py exposes /upload, /query, /list and /details/{doc_id}. Text is extracted with PyPDF2/python-docx/openpyxl, embedded via backend/services/rag_service.py (Sentence-Transformers + FAISS index persisted under ./data/faiss_indexes), and summarised/answered via backend/services/analyzer_service.py using Gemini.

Technologies: FastAPI, SQLAlchemy, PyPDF2, python-docx, openpyxl, sentence-transformers, faiss-cpu, google-generativeai.

10.9 General AI Construction Assistant Module

Purpose: A multimodal chat assistant supporting text, uploaded images/documents, and voice input/output.

Inputs: Text prompts, optional uploaded image, optional uploaded audio file.

Outputs: Text reply (optionally read back as synthesised speech audio), persisted chat history per session.

Workflow: backend/routers/ai.py exposes /general-chat and history endpoints; audio transcription and synthesis are handled by backend/utils/audio.py (SpeechRecognition for STT, gTTS for TTS, with generated audio files served from the /cache static mount).

Technologies: FastAPI, google-generativeai, SpeechRecognition, gTTS, Pillow.

Modules explicitly referenced in the FastAPI application description or the project's design specification but not found implemented in the code — Login/Registration as a distinct "module" beyond the Authentication router, a dedicated Admin Module, and a Hiring/Recruitment module — are addressed in Section 19 (Disadvantages).

11. Frontend Features

The Streamlit frontend (frontend/app.py, frontend/pages/*.py, frontend/components/dashboard_renderers.py) implements the following UI elements, confirmed from source:

- Landing / authentication screen with login, registration and OTP-verification states managed through Streamlit session state (frontend/app.py).
- Sidebar branding and settings panel ("Hub Settings") used for navigation between pages.
- Role-aware dashboard cards and statistics rendered via frontend/components/dashboard_renderers.py.
- Data-entry forms for logging manual sensor readings, incidents, tasks and inventory consumption.
- Interactive Plotly charts and tables across all twelve monitoring/inventory/telematics/schedule pages.
- File-upload components for the document analyzer (PDF/DOCX/XLSX) and the AI assistant (images, audio).
- Chat-style conversational UI for the AI construction assistant and safety chatbot.
- Custom CSS styling (frontend/assets/style.css) layered on top of Streamlit's default component set.

Not confirmed in the current code: a dedicated notifications widget component, dark/light theme toggle CSS file, or a maps page — these appear in the project's design specification but no corresponding frontend file exists in the repository.

12. Backend Features

12.1 APIs

Seventeen FastAPI routers are registered in `backend/main.py`, covering authentication, dashboard, AI assistant, document analysis, safety, safety monitoring, schedule, telematics, inventory, waste, noise, air quality, water, structural health, energy, and weather — each exposing a REST-style set of GET/POST endpoints documented automatically at the FastAPI /docs Swagger UI.

12.2 Authentication

Stateless JWT bearer-token authentication is implemented via `backend/utls/security.py` (token creation/verification) and enforced per-route through the `get_current_user` FastAPI dependency, which decodes the token and loads the corresponding SQLAlchemy User row.

12.3 Business Logic

Business logic is separated from the HTTP layer into a `backend/services/` package (one service module per feature area), keeping routers focused on request/response handling and validation.

12.4 AI Processing

AI processing is centralised in `backend/services/ai_service.py`, `backend/services/analyzer_service.py` and `backend/services/rag_service.py`, all of which call the Google Gemini API through the `google-generativeai` SDK when a Gemini API key is configured in `.env`.

12.5 Database Operations

CRUD operations are implemented with SQLAlchemy ORM sessions injected via the `get_db` FastAPI dependency defined in `backend/database.py`, which also provides optional async MongoDB (Motor) and Redis clients for production use.

12.6 Session Management

Sessions are stateless on the backend (JWT-based); on the frontend, Streamlit's `st.session_state` persists the access token, role, and profile for the duration of the browser session.

12.7 File Processing

Uploaded files are processed in-memory for text extraction (PyPDF2, python-docx, openpyxl) in the document analyzer and AI assistant routers, and images are processed with Pillow for the simulated PPE-detection feature.

13. AI Features

13.1 Large Language Model (LLM) Integration

Google Gemini (via google-generativeai) is used directly — not through LangChain, despite langchain and its related packages being listed in requirements.txt; no LangChain imports were found anywhere in the codebase. Gemini powers the general AI assistant, the safety chatbot, and document summarisation/risk extraction.

13.2 Retrieval-Augmented Generation (RAG) & Embeddings

backend/services/rag_service.py builds a FAISS vector index from Sentence-Transformers embeddings (default model: all-MiniLM-L6-v2) of uploaded document text, enabling semantic retrieval of relevant chunks that are passed to Gemini as context when answering a user's question about a specific document.

13.3 OCR / Document Parsing

Text extraction from uploaded files uses PyPDF2 for PDFs, python-docx for Word documents, and openpyxl for Excel spreadsheets (backend/routers/ai.py). No true image-based OCR (e.g. pytesseract or Tesseract binary invocation) was found wired into the code, despite pytesseract and pdf2image being listed as dependencies and a TESSERACT_CMD variable being defined in configuration.

13.4 Computer Vision

The PPE-compliance detection endpoint (backend/services/detection_service.py) does not perform real object detection. It is a simulation that uses Pillow (PIL) to draw illustrative bounding boxes and generate a plausible compliance score and detected/missing item list. The ultralytics (YOLO) package listed in requirements.txt is not imported or used anywhere in the code.

13.5 Voice AI

Speech-to-text transcription (backend/utils/audio.py, using the SpeechRecognition library) and text-to-speech synthesis (gTTS) are implemented and wired into the general AI assistant router, allowing spoken input and audible responses.

14. Database Design

The default and fully wired data store is SQLite, accessed through SQLAlchemy ORM models. A MongoDB connection (via Motor) and a Redis client are also implemented in backend/database.py for a production deployment, but no MongoDB collections/documents were found defined in the codebase beyond the connection helper — MongoDB support is present at the infrastructure level but not used by any router in the current implementation.

14.1 Core Tables (SQLite / SQLAlchemy)

Table	Key Fields	Purpose
users	id (PK), email (unique), password hash, role, timestamps	Stores registered accounts and their selected role
task_items	id (PK), user_id (FK → users.id), title, is_completed	Dashboard to-do items
activity_log / system_notifications	id (PK), user_id/role, message, timestamp	Dashboard activity feed and role/user-targeted notifications
general_chat_history	id (PK), session_id, message, response, timestamp	AI assistant chat history per session
analyzed_documents	id (PK), filename, upload metadata, vector index reference	Tracks uploaded documents and their FAISS index
safety_incidents	id (PK), reporter_id (FK → users.id), description, timestamp	Logged safety incidents
schedule_tasks	id (PK), name, dates, CPM attributes	Project schedule tasks used for delay prediction
equipment_telemetry	id (PK), name, health indicators	Simulated equipment telemetry readings
material_stocks / material_orders	id (PK), material_name (unique), quantity, order status	Inventory stock and reorder tracking
waste_logs / waste_goals	id (PK), waste_type, quantity, goal thresholds	Waste tracking and sustainability targets
noise_logs / noise_configs	id (PK), sensor_name, decibel reading, threshold	Noise monitoring readings and thresholds
air_quality_logs / air_quality_configs	id (PK), sensor_name, gas readings, threshold	Air-quality readings and safety thresholds
water_logs / water_configs	id (PK), sensor_name, flow reading, threshold	Water flow/leakage readings and thresholds
structural_logs / structural_configs	id (PK), sensor_name, stress reading, threshold	Structural health readings and thresholds
energy_logs / energy_configs	id (PK), sensor_name, consumption reading, threshold	Energy consumption readings and thresholds
weather_logs /	id (PK), sensor_name, weather	Weather hazard readings and

Table	Key Fields	Purpose
weather_configs	reading, threshold	thresholds

14.2 Relationships

task_items and safety_incidents each hold a foreign key back to users.id, linking records to the user who created them. The remaining monitoring tables are largely independent per-sensor logs keyed by sensor_name rather than being linked to a separate "site" or "project" entity — the current schema assumes a single implicit site/project context.

14.3 CRUD Operations

Every monitoring and inventory router implements Create (POST /log, /consume, /reorder), Read (GET /logs, /configs, /stocks, /orders), Update (POST /config, /update-delivery), and a bulk Delete/Reset-to-default operation (POST /reset), giving each module full CRUD coverage.

15. Workflow

End-to-end operational workflow, confirmed against the router and service code:

17. User Registration & OTP Verification — user submits registration form; backend generates and emails an OTP; user verifies it to activate the account.
18. Login — user submits credentials; backend validates the password hash and issues a JWT access token.
19. Dashboard Load — Streamlit calls /dashboard/stats with the bearer token; the backend returns role-aware statistics, tasks and notifications.
20. Module Interaction — user navigates to a monitoring, inventory, schedule, telematics, or safety page and either logs data manually or triggers a simulation.
21. AI Processing (where applicable) — for the assistant and document analyzer, the request is forwarded to Google Gemini (with FAISS retrieval for document Q&A) and the response is displayed, optionally as synthesised speech.
22. Persistence — all logged data, chat history, and configuration changes are written to the SQLite database through SQLAlchemy sessions.
23. Output — updated charts, tables, and AI responses are rendered back in the Streamlit UI for the user.

16. Security Features

- Authentication: JWT bearer tokens (python-jose) issued at login and required on protected endpoints via the `get_current_user` dependency.
- Password Encryption: passwords are hashed with `bcrypt` via `passlib`'s `CryptContext` before storage — plain-text passwords are never persisted.
- OTP Verification: registration and password reset require a one-time code sent by email, adding a second verification step.
- Session Handling: the frontend never stores the raw password; only the JWT token is kept in Streamlit's session state and attached as an Authorization header.
- API Security: CORS is restricted to a known list of local development origins in `backend/main.py`.
- Data Validation: Pydantic schemas validate all incoming request payloads (e.g. `UserRegister`, `UserLogin`) before they reach business logic.
- Rate limiting configuration (`RATE_LIMIT_LOGIN_MAX`, `RATE_LIMIT_LOCKOUT_MINUTES`) is defined in settings, though the specific lockout enforcement logic should be verified in `backend/services/auth_service.py` before relying on it in production.

Limitation observed: the user's role field personalises dashboard content and chatbot tone, but no endpoint was found that rejects a request based on role (i.e. there is no enforced role-based access control / authorization layer — `backend/middleware/` contains only an empty `__init__.py`). Any authenticated user can currently call any endpoint regardless of their selected role.

17. Outcomes

17.1 Expected Results

A working full-stack demonstration where a user can register, log in, view a personalised dashboard, generate/inspect simulated site-monitoring data across six domains, manage inventory and equipment, and interact with an AI assistant capable of understanding uploaded documents and images.

17.2 Actual Implementation

All of the above is present and functional in the codebase, backed by 72 automated test functions across 16 test files (tests/test_*.py) covering authentication, dashboard, and every monitoring/AI module.

17.3 User Benefits

- A single dashboard replacing multiple spreadsheets and paper logs.
- Instant AI-generated summaries and Q&A over uploaded project documents.
- Configurable safety thresholds per monitoring domain.

17.4 Business Benefits

- Reduced manual data-entry overhead through simulation-assisted demonstration and structured logging.
- Faster document review through AI summarisation, reducing time spent reading lengthy reports.

17.5 Technical Achievements

- Clean separation of frontend, backend routers, service layer and ORM models.
- Working RAG pipeline combining FAISS, Sentence-Transformers and Gemini.
- Consistent, repeatable CRUD pattern applied across six independent monitoring modules.

18. Advantages

- Consolidates six site-monitoring domains, inventory, telematics, scheduling, and safety into a single authenticated application.
- Rapid Python-only development stack (FastAPI + Streamlit) with no separate frontend build pipeline.
- AI-powered document analyzer saves time reading long project reports.
- Retrieval-Augmented Generation grounds AI answers in the user's own uploaded documents rather than relying purely on the model's memory.
- Multimodal AI assistant supports text, image, and voice input/output.
- Consistent, predictable REST API structure across monitoring modules (logs/configs/log/simulate/config/reset) that is easy to extend.
- Password hashing and JWT-based authentication protect user credentials and sessions.
- OTP-based email verification adds a second authentication factor at registration and password reset.
- Configurable safety thresholds per monitoring domain allow adaptation to different site standards.
- Automatic Swagger/OpenAPI documentation is generated for free by FastAPI at /docs.
- Automated pytest suite (72 test functions) provides regression protection across modules.
- Graceful degradation: Redis cache failures are caught and logged without crashing the application.
- Supports both a lightweight SQLite database for development and a MongoDB connection for production scale-out.
- Role selection allows dashboard content and AI chat tone to be personalised per user type.
- Simulation endpoints allow the system to be demonstrated meaningfully without physical IoT sensor hardware.

19. Disadvantages

- No enforced role-based access control — any authenticated user can call any endpoint regardless of their selected role.
- All environmental and structural sensor readings are simulated with randomised values rather than sourced from real IoT hardware.
- The PPE-compliance detection endpoint simulates computer-vision output using Pillow rather than running a real trained YOLO/CV model, despite ultralytics being listed as a dependency.
- LangChain and related packages are listed in requirements.txt but are not imported or used anywhere in the code; Gemini is called directly.
- pytesseract / pdf2image (true OCR of scanned/image-based documents) are listed as dependencies but no OCR call was found in the code — only text-layer extraction from PDFs/DOCX/XLSX is implemented.
- reportlab, qrcode, folium and APScheduler are listed as dependencies but no usage of them was found in the code, suggesting planned-but-unbuilt features (PDF export, QR codes, map views, and scheduled background jobs).
- The hiring/recruitment functionality implied by the backend's API description ("Intelligent Hiring Platform") is not implemented anywhere in the code.
- MongoDB and Redis integrations exist at the connection level but are not used by any router — the application currently only persists data to SQLite.
- The .env file bundled with the analysed archive contains what appear to be live configuration values and should never be committed to a public repository or shared as-is.
- AI features (assistant, document analysis, RAG) depend on a valid, quota-available Gemini API key; without one, those modules cannot function.
- The monitoring modules are not tied to a distinct "project" or "site" entity, so multi-site deployments are not currently supported by the schema.
- No dedicated frontend notification widget, theming toggle, or map page were found, despite being referenced in the project's own design specification.

20. Challenges Faced During Development

The following challenges are inferred from the structure of the codebase (e.g. graceful fallback logic, simulation-based modules, and dual database support) rather than from an explicit changelog, since no development log was included in the archive.

20.1 Frontend

Coordinating Streamlit's rerun-on-interaction model with persistent authentication state required careful use of `st.session_state`, visible in the repeated session-state initialisation block at the top of `frontend/app.py`.

20.2 Backend

Maintaining a consistent CRUD pattern across six near-identical monitoring routers (air quality, noise, water, structural, weather, energy) while keeping each domain's specific fields (e.g. decibel levels vs. gas concentrations) required disciplined code organisation, achieved here by mirroring router/model/service triads.

20.3 Database

Supporting both a synchronous SQLAlchemy/SQLite path and an asynchronous Motor/MongoDB path in the same `database.py` module required separate connection-handling logic for each, plus defensive Redis connection code that does not crash the app if Redis is not running.

20.4 AI Integration

Integrating Gemini for both direct chat and Retrieval-Augmented Generation required building a custom FAISS-based retrieval layer (`backend/services/rag_service.py`) since LangChain — despite being installed — was not ultimately used for chain orchestration.

20.5 APIs

Keeping the Streamlit `api_client.py` in sync with 17 separate FastAPI routers as functionality grew is a common source of integration drift; the project mitigates this with a single centralised `APIClient` class.

20.6 Deployment

Running two separate processes (FastAPI backend and Streamlit frontend) on different ports required a coordinated startup script (`run.bat`) and explicit CORS configuration.

20.7 Testing

Testing AI-dependent endpoints (Gemini, FAISS) without always having live API access is addressed by the project's test databases (`tests/test_*.db` files) that isolate test runs from the main development database.

20.8 Performance

Loading the Sentence-Transformers embedding model can be slow on first use; the code addresses this with a lazily-initialised, module-level `_embedding_model` variable in `rag_service.py` so it is loaded once and reused.

20.9 User Interface / Debugging

Because Streamlit re-executes the whole script on every interaction, debugging UI state issues (e.g. why a chart didn't refresh) is generally harder than in a persistent single-page JavaScript application — a known trade-off of the framework choice discussed in Section 9.

21. Testing

The repository includes a tests/ directory with 16 test files and 72 test functions built on pytest, pytest-asyncio and httpx, covering the backend API layer.

Test File	Area Covered
test_auth.py	Registration, OTP verification, login, logout, password reset
test_dashboard.py	Dashboard statistics, tasks, activities, chat
test_ai.py	General AI assistant chat and history endpoints
test_document_analyzer.py	Document upload, RAG query, document listing
test_safety.py / test_safety_monitoring.py	Safety chatbot, incidents, checklist, PPE detection
test_schedule.py	Schedule tasks and delay prediction
test_telematics.py	Equipment telemetry and maintenance scheduling
test_inventory.py	Stock consumption and reorder workflow
test_waste.py, test_noise.py, test_air_quality.py, test_water.py, test_structural.py, test_energy.py, test_weather.py	CRUD + simulate + config + reset for each monitoring module

21.1 Testing Types Applied

- Unit Testing: individual service-layer functions (e.g. OTP generation, password hashing) are exercised indirectly through endpoint tests.
- Integration Testing: httpx-based API client tests call full FastAPI request/response cycles against a dedicated per-module SQLite test database.
- Functional Testing: each router's documented behaviour (log, simulate, config, reset) is tested end-to-end.

21.2 Testing Types Not Found in the Repository

No dedicated UI testing (e.g. Selenium/Playwright for the Streamlit frontend), load/performance testing, or formal user-acceptance-testing artefacts were found in the archive; these would need to be added for a production-readiness assessment.

22. Future Enhancements

24. Implement genuine role-based access control (authorization), not just role-based UI personalisation.
25. Replace simulated sensor readings with real IoT hardware integration (MQTT/HTTP ingestion from physical sensors).
26. Wire up a real object-detection model (e.g. an actual YOLOv8 model via ultralytics) for PPE compliance checking instead of the current simulated detector.
27. Add true OCR (pytesseract) support for scanned or image-based PDF documents.
28. Adopt LangChain (already listed as a dependency) to formalise the RAG and chat orchestration pipeline.
29. Complete the MongoDB and Redis integration so production deployments can move off SQLite.
30. Build the hiring/recruitment module implied by the backend's API description.
31. Introduce a proper project/site entity so the platform can support multiple concurrent construction sites.
32. Add PDF/QR-code report export using the already-installed reportlab and qrcode libraries.
33. Add a map-based site view using folium, which is installed but currently unused.
34. Introduce scheduled background jobs (e.g. daily digest emails, automatic threshold-breach alerts) using APScheduler, which is installed but currently unused.
35. Add push/email notification delivery when a configured safety threshold is breached.
36. Add automated UI testing for the Streamlit frontend.
37. Add rate-limiting enforcement middleware to back the already-configured RATE_LIMIT_LOGIN_MAX setting.
38. Introduce refresh tokens to complement the current JWT access-token flow.
39. Add audit logging for administrative actions.
40. Containerise the application (Docker Compose) for consistent multi-service (API + frontend + MongoDB + Redis) local and cloud deployment.
41. Add CI test execution to the existing .github/workflows directory (currently present but empty in the analysed archive).
42. Add multilingual support for the AI assistant and UI.
43. Introduce a mobile-responsive or dedicated mobile client, since Streamlit's layout is primarily desktop-oriented.

23. Applications

- Academic: demonstrates integration of a Python web stack with generative AI (LLM + RAG) for a capstone/final-year engineering project.
- Industrial: as a prototype, the monitoring and inventory modules illustrate a pattern that could be extended for small-to-mid-sized construction contractors.
- Commercial: the document analyzer and AI assistant patterns are reusable for any business needing AI-assisted document review.
- Government: the safety-incident and threshold-configuration pattern could support compliance reporting for regulated worksites, subject to the access-control improvements noted in Section 22.
- Enterprise: the modular router/service/model pattern is a reusable template for adding further operational domains beyond construction.
- Research: the RAG implementation (FAISS + Sentence-Transformers + Gemini) is a useful reference example for combining open-source embeddings with a hosted LLM.

24. Conclusion

Construction Intelligent Hub is a functional, full-stack demonstration of an AI-assisted construction site management platform, combining a FastAPI backend, a Streamlit frontend, and Google Gemini-powered AI features across authentication, dashboards, six environmental/structural monitoring domains, inventory, telematics, scheduling, safety, and document analysis. The codebase shows disciplined, repeatable patterns (router/service/model triads) and a working Retrieval-Augmented Generation pipeline, backed by a 72-test automated suite. At the same time, several dependencies declared in requirements.txt — LangChain, YOLO/ultralytics, pytesseract OCR, reportlab, qrcode, folium and APScheduler — are not yet wired into working features, and role-based authorization is not enforced, indicating clear, concrete directions for continued development as outlined in Section 22. As it stands, the project is best characterised as a robust functional prototype rather than a production-hardened system, and it successfully fulfils its stated objectives within that scope.

25. References

- FastAPI — <https://fastapi.tiangolo.com/>
- Streamlit — <https://docs.streamlit.io/>
- SQLAlchemy — <https://docs.sqlalchemy.org/>
- Motor (MongoDB async driver) — <https://motor.readthedocs.io/>
- Redis-py — <https://redis.readthedocs.io/>
- python-jose — <https://python-jose.readthedocs.io/>
- Passlib — <https://passlib.readthedocs.io/>
- Pydantic — <https://docs.pydantic.dev/>
- Google Generative AI (Gemini) — <https://ai.google.dev/>
- FAISS — <https://faiss.ai/>
- Sentence-Transformers — <https://www.sbert.net/>
- PyPDF2 — <https://pypdf2.readthedocs.io/>
- python-docx — <https://python-docx.readthedocs.io/>
- openpyxl — <https://openpyxl.readthedocs.io/>
- Pillow — <https://pillow.readthedocs.io/>
- SpeechRecognition — <https://pypi.org/project/SpeechRecognition/>
- gTTS — <https://gtts.readthedocs.io/>
- Plotly — <https://plotly.com/python/>
- Pandas — <https://pandas.pydata.org/docs/>
- NumPy — <https://numpy.org/doc/>
- pytest — <https://docs.pytest.org/>

27. Project Folder Structure

Simplified tree of the analysed repository (auto-generated files such as `__pycache__`, `.pytest_cache` and cached audio/index files omitted for readability):

Hemanthinfo/	
├── backend/	FastAPI application
│ ├── main.py	App factory, router registration, CORS, lifespan/init_db
│ ├── config.py	Pydantic Settings loaded from <code>.env</code>
│ ├── database.py	SQLite (SQLAlchemy), MongoDB (Motor), Redis connections
│ ├── models/	17 SQLAlchemy ORM model files (one per data entity)
│ ├── routers/	17 FastAPI routers (one per feature area)
│ ├── schemas/	Pydantic request/response schemas
│ ├── services/	Business logic layer (18 service modules)
│ └── utils/	<code>security.py</code> (JWT/bcrypt), <code>audio.py</code> (STT/TTS)
├── frontend/	Streamlit application
│ └── app.py	Entry point, session state, auth screens, custom CSS
├── loader	
│ └── pages/	14 feature pages (monitoring, inventory, safety, AI,
etc.)	
│ ├── components/	<code>dashboard_renderers.py</code>
│ ├── utils/api_client.py	Centralised HTTP client to the backend
│ └── assets/style.css	Custom styling
├── ai/	Scaffolded chains/prompts/vector_store packages (empty
stubs)	
├── data/	SQLite databases, FAISS indexes, cached TTS audio
(runtime data)	
├── tests/	16 pytest files, 72 test functions
├── .kiro/specs/	Original design/requirements/tasks specification
documents	
├── .env / .env.example	Runtime secrets / documented configuration template
├── requirements.txt	Python dependency list
└── run.bat	Windows startup script for backend + frontend